Java Silver（SE 17・1Z0-825-JPN）　5 継承とインタフェースの使用　#12

インタフェースとシール・クラス

「何ができるか」の約束を型にする。継承してよい相手を自分で決める

各ページの下に、根拠にした Oracle の資料（JLS 8.1.1.2・8.1.6・8.4.8・9.1・9.3・9.4・15.12、JEP 409）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 5（インタフェースの作成と実装、デフォルト・メソッドとプライベート・メソッド、シール・クラス）

1 インタフェースの基本
フィールドは public static final、メソッドは public abstract が自動で付く

2 default メソッド
中身のある実装。衝突したらクラスでオーバーライド、I.super で呼ぶ

3 static・private メソッド
static はインタフェース名で。private は中だけで

4 シール・クラス
sealed と permits で直接の子を限定。子は final・sealed・non-sealed

	クラス	インタフェース
親を何個	extends 1 つ	extends 複数（インタフェースどうし）
クラスが実装	—	implements 複数
new	できる（abstract 以外）	できない
フィールド	自由	定数だけ
コンストラクタ	持てる	持てない

Java 8 で default・static、Java 9 で private、Java 17 で sealed が正式に。試験は Java 17 の範囲。

出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト／JEP 409: Sealed Classes（Java 17 で正式）／JLS 9

第1章

インタフェースの基本

書かなくても修飾子が付く。実装する側は public で。

インタフェースに書けるものと、自動で付く修飾子

```
interface Shape {
  int SIDES = 0;                  // public static final
  double area();                   // public abstract
  default String info() {          // public
    return label("area") + area();
  }
  static Shape unit() { … }        // public static
  private String label(String s) { return s + "="; }
}
```

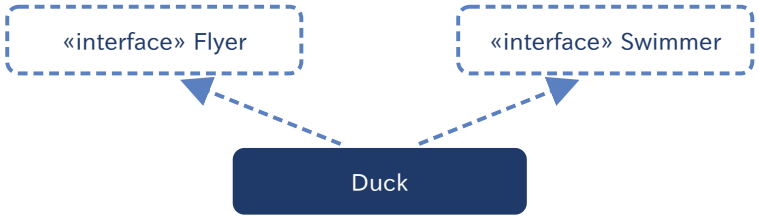
… は省略。各行の右の注釈は、書かなくても付く修飾子（Claude が作成）。

書き方	JDK 17
`int N;`（初期値なし）	= expected
`I.N = 20;`	cannot assign a value to final variable N
`void m() { }`（default なし）	interface abstract methods cannot have body
`protected void m();`	modifier protected not allowed here
`final void m();`	modifier final not allowed here
`new I()`	I is abstract; cannot be instantiated

アクセス修飾子は public と private だけ。何も書かなければ public。クラスの「修飾子なし＝パッケージアクセス」とはちがう。

出典：JLS 9.3（Every field declaration in the body of an interface delaration is implicitly public, static, and final）・9.4（If no access modifier is given, the method is implicitly public／An interface method lacking a private, default, or static modifier is implicitly abstract）／結果は JDK 17 で確かめたもの

implements：複数実装と、実装の決まり



点線の矢印 = implements Flyer, Swimmer

```
class Duck implements Flyer, Swimmer {
  public String fly()  { return "fly"; }
  public String swim() { return "swim"; }
}
Flyer f = new Duck();
Swimmer s = (Swimmer) f;    // OK（中身は Duck）
// f.swim() は cannot find symbol（Flyer 型には swim がない）
```

実装クラスの書き方（I に void m(); void n();）	JDK 17
`void m() { }`（public なし）	m() in C cannot implement m() in I ／ attempting to assign weaker access privileges; was public
m() だけ実装した普通のクラス	C is not abstract and does not override abstract method n() in I
m() だけ実装した `abstract` クラス	OK（残りはサブクラスで）
`interface AB extends A, B`	OK（インタフェースは複数 extends）

> extends と implements を両方書くときは `class C extends P implements I, J` の順。

出典：JLS 8.1.5（The optional implements clause in a class declaration specifies the direct superinterface types of the class being declared）／JLS 9.1.3（the interface being declared extends each of the specified interface types）／結果は JDK 17 で確かめたもの

インタフェース型へのキャスト

コード（Flyer はインタフェース）	結果（JDK 17）
`(Swimmer) flyer`（中身が両方を実装）	OK
`(Flyer) dog`（Dog は final でなく、Flyer を実装しない）	コンパイルは通り、実行時に ClassCastException
`(Flyer) cat`（Cat は final で、Flyer を実装しない）	incompatible types: Cat cannot be converted to Flyer
`obj instanceof I`（中身が I を実装）	true

理由：final でない Dog には「Flyer を実装したサブクラス」があるかもしれない。だからコンパイラは止めず、実行時に調べる。final の Cat にはサブクラスがないので、ありえないと判断できる。



出典：JLS 5.1.6.3（If T is an interface type, then R must implement interface T (§8.1.5), or a ClassCastException is thrown）／JLS 15.16／結果は JDK 17 で確かめたもの

第2章

default・static・private メソッド

インタフェースにも中身を書ける。ただし呼び方と継承のされ方がちがう。

default メソッド：中身付きの実装

```
interface Greeter {
  String name();
  default String greet() { return "Hello, " + name(); }
}
class En implements Greeter {
  public String name() { return "Ann"; }
}
class Jp implements Greeter {
  public String name() { return "Ken"; }
  public String greet() { return "Konnichiwa, " + name(); }
}
// new En().greet() + " / " + new Jp().greet()
```

```
Hello, Ann / Konnichiwa, Ken
```

書き方	JDK 17
`default void m();`（中身なし）	missing method body, or declare abstract
`default String toString() { … }`	default method toString in interface I overrides a member of java.lang.Object
`static default void m() { }`	illegal combination of modifiers: static and default
実装クラスで public なしでオーバーライド	weaker access privileges

default メソッドはインスタンスメソッド。実装クラスに継承され、オーバーライドできる。中から抽象メソッド（name()）を呼べば、実装クラスのものが動く。

出典：JLS 9.4（A default method is an instance method declared in an interface with the default modifier. Its body is always represented by a block, which provides a default implementation for any class that implements the interface without overriding the method）・9.4.1.2（It is a compile-time error if a default method is override-equivalent (§8.4.2) with a non-private method of the class Object）／結果は JDK 17 で確かめたもの

default の衝突と、インタフェース名.super

衝突する（Claude が作成）

```
interface I { default String hi() { return "I"; } }
interface J { default String hi() { return "J"; } }
class C implements I, J { }
```

```
error: types I and J are incompatible;
  class C inherits unrelated defaults for hi() from types I and J
```

C でオーバーライドして解決

```
class C implements I, J {
  public String hi() {
    return I.super.hi() + J.super.hi() + "C";
  }
}   // new C().hi() → IJC
```

呼び方（C implements I）	意味・結果
`I.super.hi()`	I の default メソッド
`super.hi()`	スーパークラス（Object）を探す → cannot find symbol
`I.hi()`	non-static method hi() cannot be referenced from a static context

> **implements に書いた順番では決まらない。**関係のない 2 つのインタフェースの default が重なったら、クラスで必ずオーバーライドする。

I.super で呼べるのは、そのクラスが直接 implements しているインタフェースだけ。

出典：JLS 8.4.8.4（It is a compile-time error if a class C inherits a default method whose signature is override-equivalent with another method inherited by C …）／JLS 15.12.1（It is a compile-time error if I is not a direct superinterface of E）／結果は JDK 17 で確かめたもの

どのメソッドが使われる？3 つの決まり

① クラスが優先
スーパークラスの具体的なメソッド ＞ インタフェースの default

② より具体的なインタフェースが優先
J extends I で J がオーバーライド → J の default

③ それでも決まらなければエラー
無関係な I と J の default → クラスでオーバーライド

コード（I・J は default hi() または m() を持つ）	結果（JDK 17）
`class P { public String hi() … }` `class C extends P implements I { }`	P（クラスが優先）
P の hi() が public でない	hi() in P cannot implement hi() in I
`interface J extends I`（J も m() を default で書く） `class C implements I, J { }`	J
`interface J extends I { void m(); }` `class C implements J { }`	C is not abstract and does not override abstract method m() in J

最後の行：子のインタフェースが default を抽象メソッドに戻すと、実装クラスは中身を書かなければならない。

出典：JLS 8.4.8（No concrete method inherited by C from its direct superclass type has a signature that is a subsignature of the signature of m as a member of D）／結果は JDK 17 で確かめたもの

static メソッドと private メソッド

```
interface Logger {
  default void info(String m) { log("INFO", m); }
  default void warn(String m) { log("WARN", m); }
  private void log(String lv, String m) {   // 共通処理
    System.out.println("[" + lv + "] " + m);
  }
}
interface Calc {
  static int twice(int x) { return helper(x) * 2; }
  private static int helper(int x) { return x + 1; }
}
// Calc.twice(3) → 8
```

呼び方（C implements I、x は I 型）	JDK 17
`I.s()`（static）	OK
`x.s()`	illegal static interface method call
`C.s()`	cannot find symbol（継承されない）
ほかのクラスから private の `log()`	cannot find symbol
static から private（static なし）	non-static method p() cannot be referenced from a static context
static から private static	OK

クラスの static との差：クラスの static は変数やサブクラス名でも呼べるが、インタフェースの static はインタフェース名だけ。

出典：JLS 15.12.3（the compile-time declaration must not be a static method declared in an interface, or a compile-time error occurs）／JLS 8.4.8（A class does not inherit private or static methods from its superinterface types）／JLS 9.4（It is permitted for an interface method declaration to contain both private and static）／結果は JDK 17 で確かめたもの

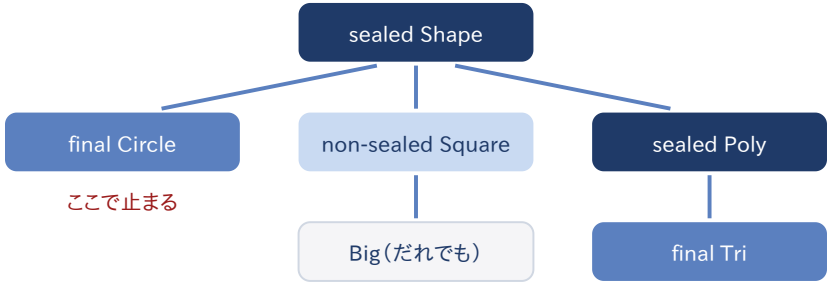
第3章

シール・クラス

直接の子を permits で決め、子にはどこまで開くかを書かせる。

sealed・permits と 3 つの修飾子

```
sealed class Shape permits Circle, Square, Poly { }
final class Circle extends Shape { }
non-sealed class Square extends Shape { }
sealed class Poly extends Shape permits Tri { }
final class Tri extends Poly { }
class Big extends Square { }     // OK
```



子の修飾子	意味
final	ここで止める。継承できない
sealed	さらに permits で子を限定する
non-sealed	ここから下はだれでも継承できる

> **permits が限定するのは「直接の」子だけ。**Big は Square の子なので Shape の permits に書かなくてよい。new Big() を Shape 型に入れて instanceof Square は true。

レコードと enum は暗黙に final なので、修飾子なしで許可された子になれる。

出典：JLS 8.1.1.2（It is a compile-time error if a class has a sealed direct superclass or a sealed direct superinterface, and is not declared final, sealed, or non-sealed either explicitly or implicitly）／JEP 409／結果は JDK 17 で確かめたもの

シール・クラスのコンパイルエラー一覧

書き方	JDK 17 のメッセージ
許可した子に final・sealed・non-sealed がない	sealed, non-sealed or final modifiers expected
permits にないクラスが extends	class is not allowed to extend sealed class: Shape (as it is not listed in its permits clause)
sealed がないのに permits	invalid permits clause ／ (class must be sealed)
permits に書いたクラスが extends していない	invalid permits clause ／ (subclass Circle must extend sealed class)
permits に同じクラスを 2 回	invalid permits clause ／ (must not contain duplicates: Circle)
permits を省略し、同じファイルに子がない	sealed class must have subclasses
`sealed final class`・`final non-sealed class`	illegal combination of modifiers: final and sealed（final and non-sealed）
親が sealed でないのに non-sealed	non-sealed modifier not allowed here
シール・インタフェースを継承するインタフェースに何もない	sealed or non-sealed modifiers expected（インタフェースに final はない）

エラーにならないもの：permits を省略して同じファイルに子を書く。abstract でないシール・クラスを new する。sealed abstract class。

出典：JLS 8.1.6（It is a compile-time error if a class declaration has a permits clause but no sealed modifier／It is a compile-time error if the same class is specified more than once in a permits clause／It is a compile-time error if the declaration of a sealed class C lacks a permits clause and C has no permitted direct subclasses）／結果は JDK 17 で確かめたもの

シール・インタフェースとレコード

```
sealed interface Expr permits Num, Add { }
record Num(int v) implements Expr { }   // 暗黙に final
final class Add implements Expr {
  final Expr l, r;
  Add(Expr l, Expr r) { this.l = l; this.r = r; }
}
static int eval(Expr e) {
  if (e instanceof Num n) return n.v();
  if (e instanceof Add a) return eval(a.l) + eval(a.r);
  throw new IllegalStateException();
}
// eval(new Add(new Num(2), new Add(new Num(3), new Num(4))))
```

```
9
```

シール・インタフェースの子	付けるもの
実装するクラス	final・sealed・non-sealed
実装するレコード	なし（暗黙に final）
継承するインタフェース	sealed・non-sealed
permits にないクラスが implements	class is not allowed to extend sealed class: I …

種類が決まっている値に向く。式の木・図形・支払い方法など。Java 17 では if と instanceof で分ける。

switch でパターンを使い、全種類を網羅したかをコンパイラに調べさせる書き方は、Java 17 ではプレビュー（Java 21 で正式）。

出典：JLS 9.1.1.4（It is a compile-time error if an interface has a sealed direct superinterface and is not declared sealed or non-sealed）／JLS 8.10（A record class is implicitly final）／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
実装クラスがコンパイルできるか	public を付けたか。抽象メソッドを全部書いたか（abstract クラスなら不要）
インタフェースの定数	public static final。代入はエラー。I.N・C.N・変数.N で読める
default が 2 つ重なる	無関係なら types I and J are incompatible。オーバーライドして I.super.hi()
クラスと default が重なる	クラスが優先。public でないとエラー
static・private の呼び方	static は I.s() だけ。private は同じインタフェースの中だけ
シール・クラスの子	直接の子は permits に書き、final・sealed・non-sealed のどれか。孫は non-sealed の下なら自由
修飾子の組み合わせ	sealed＋final、final＋non-sealed、static＋default はエラー。private＋static は OK

出典：このPDFの各ページの資料

まとめ

インタフェース
定数は public static final、メソッドは public abstract。new 不可

default
中身付き。クラス優先、具体的なほう優先、決まらなければエラー

static・private
static は I.s()。private は共通処理用、private static も可

シール・クラス
sealed＋permits。子は final・sealed・non-sealed

こんなとき	結果
実装メソッドに public を付けない	コンパイルエラー
default で toString を書く	コンパイルエラー
インタフェース型の変数で static を呼ぶ	コンパイルエラー
non-sealed の子をさらに継承	OK
レコードでシール・インタフェースを実装	OK（暗黙に final）

次は #13 例外処理。チェック例外と非チェック例外、try-catch・finally・try-with-resources・マルチキャッチへ進む。

出典：このPDFの各ページの資料